

A Tutorial on Differential Cryptanalysis

1st Mentee:Mücahit ÖZTÜRK
Gebze Technical University
m.ozturk2018@gtu.edu.tr

2nd Mentor:Hakan Önder GÜNEYİ
Middle East Technical University
hguneyi@metu.edu.tr

Abstract—In this paper, To understand the differential cryptanalysis described in the Heys article in more detail, we first studied the fundamentals of cryptanalysis, block cipher primitives, and the general logic of differential cryptanalysis. This report provides a practical theoretical explanation of differential cryptanalysis given as an example in Heys article.

Index Terms—differential, cryptanalysis, block cipher, s-box.

I. INTRODUCTION

Cryptanalysis plays a central role in evaluating the security of cryptographic algorithms by exposing their potential weaknesses. Among the most influential techniques, differential cryptanalysis provides a systematic way to study how input differences propagate through encryption and how these differences can reveal information about the secret key.

To build a clear understanding of this method, it is first necessary to review the logical operations that underlie block cipher design, with particular attention to the role of the XOR function. Block ciphers based on substitution-permutation (SP) networks, supported by components such as S-boxes and P-layers, will be considered as the primary framework.

Finally, the limitations of brute-force search will be contrasted with the efficiency of differential attacks. This report will therefore introduce the fundamental ideas behind differential cryptanalysis and outline its application to SP-network block ciphers, providing both theoretical and practical insight into the method.

A. Complexity of Computation

In the analysis of block ciphers, evaluating the complexity of an attack is essential for understanding its practicality and efficiency. The main measures used for this purpose are *time complexity*,

data complexity, and *memory complexity*. These parameters collectively provide a framework to assess whether a cryptanalytic method, such as differential cryptanalysis, is feasible against a given cipher.

Time Complexity. Time complexity refers to the computational effort required to execute an attack. It is generally expressed in terms of the number of elementary operations, such as encryptions, decryptions, or logical operations (e.g., XOR). An attack is considered practical only if its time complexity is significantly lower than that of an exhaustive brute-force search. In the context of differential cryptanalysis, time complexity is often determined by the number of chosen plaintext-ciphertext pairs that must be processed to recover the key.

Data Complexity. Data complexity measures the amount of data needed to carry out an attack successfully. In symmetric key cryptanalysis, this is typically quantified by the number of plaintext-ciphertext pairs required. A lower data complexity indicates that the attack is more realistic, since real-world adversaries are often limited in the amount of data they can obtain or manipulate. Differential cryptanalysis usually requires a substantial but structured set of chosen plaintexts, which directly affects its data complexity.

Memory Complexity. Memory complexity represents the storage resources required during an attack. This may include tables for storing intermediate values, differential probabilities, or partial subkey guesses. While memory requirements are generally less critical than time or data, excessively high memory complexity can render an otherwise efficient attack impractical. In differential cryptanalysis, lookup tables and probability distributions often play an important role in the analysis and thus contribute to the overall memory cost.

B. Some Basic Definition

Plaintext (P) refers to data or information in its original, human-readable form, which has not yet been subjected to encryption. In cryptographic systems, plaintext represents the input message that requires protection.

Key (K) in cryptography denotes a specific piece of information that, when applied within an encryption algorithm, governs the transformation of plaintext into ciphertext. Similarly, the same or a related key enables the reverse process of decryption, ensuring controlled access to the protected data.

Ciphertext (C) is the encrypted output generated by applying a cryptographic algorithm to plaintext using a key. By design, ciphertext is intended to be unintelligible without the correct key, thereby securing the confidentiality of the original message.

C. Boolean Operations in Cryptography

AND Operation. The AND operation outputs 1 only when both input bits are 1. In cryptography, it is mainly used in hardware implementations where strict input dependency is required.

OR Operation. The OR operation outputs 1 if at least one input bit is 1. Its role in cryptography is limited, as it lacks useful algebraic properties such as invertibility.

XOR Operation. The XOR (exclusive OR) operation outputs 1 when the input bits differ. Its simplicity and reversibility make it central to cryptography, particularly in block ciphers where it enables efficient mixing of data and key material.

p	q	$p \oplus q$	p	q	$p \vee q$	p	q	$p \wedge q$
0	0	0	0	0	0	0	0	0
0	1	1	0	1	1	0	1	0
1	0	1	1	0	1	1	0	0
1	1	0	1	1	1	1	1	1

Fig. 1. Truth Tables for XOR, OR, and AND Operations

Advantages of XOR Operation. One of the main reasons XOR is widely employed in cryptography is its inherent simplicity. The operation can be computed efficiently using minimal hardware or software resources, making it well suited for block cipher constructions that require repeated application of basic logical functions.

Another critical advantage of XOR is its reversibility. Unlike the OR or AND operations, XOR allows the original input to be fully recovered when the same key is applied again.

For example, consider a plaintext bit $P = 0$ and a key bit $K = 1$. Encryption is performed as $C = P \oplus K = 0 \oplus 1 = 1$. Applying the same key in the decryption process yields $P = C \oplus K = 1 \oplus 1 = 0$, thus restoring the original plaintext. This property ensures that XOR can be used both for mixing key material into the plaintext and for correctly recovering the original message during decryption.

Owing to these characteristics, XOR stands out as a fundamental component in modern block cipher design, enabling efficient, secure, and reversible transformations of data.

Uniformity and Whitening. A bitstream is *uniformly distributed* if each bit is independent and equals 0 or 1 with probability $1/2$; i.e., there is no bias or detectable pattern. When a plaintext (possibly biased) is combined with an *independent, uniformly distributed* keystream by XOR, the ciphertext becomes uniformly distributed and statistically hides the structure of the plaintext (whitening effect). Formally, for a single bit,

$$\begin{aligned}
 \Pr[C = 1] &= \Pr[P \oplus K = 1] \\
 &= \Pr[K = 0] \Pr[P = 1] + \Pr[K = 1] \Pr[P = 0] \\
 &= \frac{1}{2} \Pr[P = 1] + \frac{1}{2} (1 - \Pr[P = 1]) \\
 &= \frac{1}{2}.
 \end{aligned}$$

and similarly $\Pr[C = 0] = \frac{1}{2}$. Bitwise independence of the keystream implies independence of ciphertext bits. Moreover, for blocks, the mapping $x \mapsto x \oplus k$ is a bijection on $\{0, 1\}^n$, so a uniform input remains uniform and a uniform, independent subkey makes the output appear uniform even when the input is biased. This property helps prevent leakage of plaintext regularities and complements the confusion/diffusion provided by S-boxes and permutation layers in SP-network ciphers.

Remark. The whitening guarantee relies on the keystream (or subkeys) being independent and close to uniform. XOR of two biased, dependent sources need not be uniform; hence high-quality key material is essential.

Limitations of AND and OR Operations. Although AND and OR are fundamental Boolean functions, they exhibit significant disadvantages when considered for cryptographic use. The primary issue is their lack of reversibility: given a ciphertext output, it is not always possible to uniquely determine the original plaintext and key.

For instance, consider the AND operation. If the ciphertext is $C = 1$, then both inputs must necessarily be $P = 1$ and $K = 1$, leaving no flexibility. On the other hand, if $C = 0$, the result provides no unique information about the inputs, since any combination where at least one input is 0 produces the same output. This ambiguity makes the operation unsuitable for encryption, as decryption cannot reliably recover the plaintext.

A similar problem arises with the OR operation. If the ciphertext is $C = 0$, then both $P = 0$ and $K = 0$ must hold. However, if $C = 1$, the inputs could be $(P = 1, K = 0)$, $(P = 0, K = 1)$, or $(P = 1, K = 1)$. The output does not allow a unique inversion, which again prevents secure decryption.

These examples illustrate why AND and OR are not preferred in block cipher design. While they are useful in hardware logic and specific algorithmic contexts, their non-invertible nature makes them impractical for encryption schemes that require both security and reversibility. In contrast, the XOR operation avoids these drawbacks, ensuring both computational simplicity and guaranteed reversibility.

II. OVERVIEW OF SUBSTITUTION–PERMUTATION (SP) NETWORKS

Substitution–Permutation (SP) networks are a widely used architectural model for block ciphers. At a high level, encryption proceeds in multiple rounds, each combining nonlinearity (via substitution) and diffusion (via bit or word permutations), together with round-key mixing by XOR. This layered structure is designed to obscure statistical properties of the plaintext while spreading local changes across the entire state.

Notation. Let $P \in \{0, 1\}^n$ denote the plaintext, $C \in \{0, 1\}^n$ the ciphertext, and K the master key. A key schedule derives round keys $K_0, K_1, \dots, K_{r+1}$ from K . The state after round i is written $X^{(i)}$. The substitution layer applies a nonlinear mapping S in parallel to fixed-size bundles (e.g., m -bit S-boxes),

and the permutation layer P_{layer} reorders bits/positions to propagate dependencies. Key mixing is performed by XOR ($\oplus$).

Round structure (generic form). Many SP ciphers use optional input whitening, repeated round transformations, and a final key addition without a trailing permutation:

$$\begin{aligned} X^{(0)} &= P \oplus K_0 \quad (\text{optional input whitening}), \\ X^{(i)} &= P_{\text{layer}}(S(X^{(i-1)} \oplus K_i)), \quad i = 1, \dots, r-1, \\ X^{(r)} &= S(X^{(r-1)} \oplus K_r), \\ C &= X^{(r)} \oplus K_{r+1} \quad (\text{output whitening}). \end{aligned}$$

This template captures the standard “substitute–permute–mix” flow; specific ciphers may reorder key mixing and layer operations, but the cryptanalytic considerations remain similar.

Invertibility and decryption. Decryption applies the inverse layers in reverse order using the corresponding round keys. Invertibility follows from three facts: (i) each S-box is a bijection on $\{0, 1\}^m$, (ii) the permutation layer is a bijection on $\{0, 1\}^n$, and (iii) XOR with a fixed round key is its own inverse.

Context for analysis. For differential cryptanalysis, we will later examine how input differences propagate through S and the permutation layer across rounds, and how key mixing by XOR enables key-recovery strategies more efficiently than exhaustive search. In this subsection, our goal is only to fix the high-level algorithmic view and notation that will be used in the subsequent, more detailed sections.

A. Substitution

In the considered cipher design, the 16-bit data block is divided into four 4-bit sub-blocks. Each sub-block serves as the input to a 4×4 S-box, which performs a substitution mapping from four input bits to four output bits. Such a transformation can be efficiently implemented as a table lookup of sixteen 4-bit values, indexed by the integer representation of the input bits. The most fundamental property of an S-box is that it provides a nonlinear mapping, meaning that the output bits cannot be expressed as a linear function of the input bits. This nonlinearity is essential for introducing the confusion property in block cipher design.

In our construction, the same nonlinear mapping is used for all S-boxes across the cipher. This contrasts with the Data Encryption Standard (DES), where each round employs eight distinct S-boxes, though the same set of S-boxes is reused in every round. From the perspective of cryptanalysis, particularly linear and differential attacks, the distinction between using identical or distinct S-boxes is not critical; both settings remain vulnerable to the same forms of analysis if the S-box design itself is weak.

The mapping adopted for this cipher, presented in **Fig 2**, is taken directly from one of the S-boxes defined in DES (specifically, the first row of the first S-box). In this representation, the most significant bit in the hexadecimal notation corresponds to the leftmost input bit of the S-box.

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S[x]	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

Fig. 2. S-box Representation (in hexadecimal)

B. Permutation Layer

The permutation layer of the cipher is responsible for transposing the positions of the bits in the state. In other words, it permutes the bit positions without altering their values. This operation is essential for achieving the *diffusion* property in block cipher design: small changes in one part of the state are spread across many bits in subsequent rounds, thereby increasing the cipher's resistance to differential and linear attacks.

The specific bit permutation used in our construction is presented in Table ?? . In this table, i denotes the original bit position, and $P(i)$ denotes the new position to which the bit is moved. Bit positions are numbered from 0 (the most significant bit) to 63 (the least significant bit) in the 64-bit state.

Conceptually, the permutation can be described as follows: the output bit at position i of S-box j is connected to the input bit at position j of S-box i in the next round. This structured interconnection ensures that outputs from different S-boxes are dispersed across multiple S-box inputs in the subsequent round.

It is important to note that the permutation layer is omitted after the final round. Since the purpose of the permutation is to propagate dependencies across

rounds, applying it after the last round would not improve security but would only increase implementation cost.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P(i)$	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P(i)$	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

Bit permutation mapping $P(i)$ for the 64-bit state.

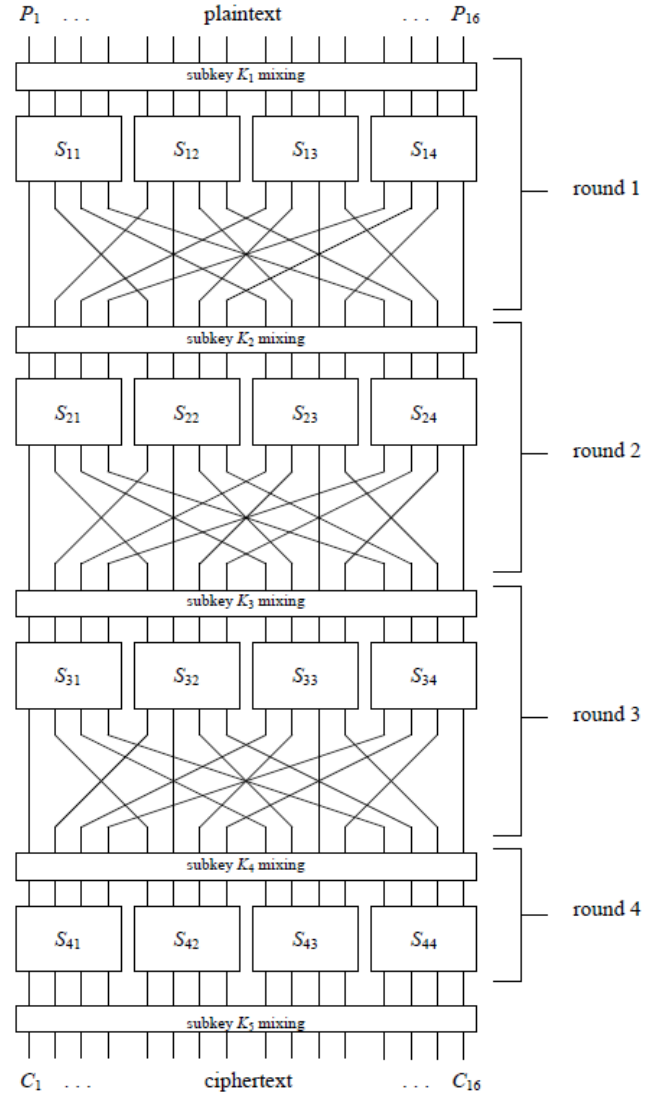


Fig. 3. Basic Substitution-Permutation Network (SPN) Cipher

C. Key Mixing

Key mixing in our cipher is achieved through a simple bit-wise exclusive-OR (XOR) between the

subkey bits associated with a given round and the data block input to that round. This operation ensures that the security of the cipher depends directly on the secrecy of the key material. Additionally, a subkey is applied after the final round to prevent cryptanalysts from simplifying attacks by working backwards through the last substitution layer.

In typical block cipher design, round subkeys are generated from the master key through a key scheduling algorithm. However, for the purposes of our cipher model, it is assumed that the subkeys are independently generated and have no mutual dependencies. This assumption simplifies the analysis, while still demonstrating the role of key mixing in providing security against exhaustive and structural attacks.

D. Decryption

Decryption in an SP-network cipher proceeds by passing the data block backward through the same layered structure as encryption. The essential difference is that the mappings used in the S-boxes during decryption are the inverses of those used in encryption. This implies that each S-box must be bijective, i.e., a one-to-one mapping with the same number of input and output bits, in order to allow decryption.

Furthermore, for correct recovery of the plaintext, the subkeys must be applied in reverse order, and their bit positions must be permuted according to the inverse of the permutation layer. Notably, the omission of a permutation after the last round in the encryption process simplifies the decryption structure, as it allows the decryption network to mirror the encryption network. This symmetry highlights an important design principle in SP-networks: maintaining structural consistency while ensuring invertibility.

E. Overall Structure of the Cipher

The complete encryption process of the considered block cipher follows a Substitution–Permutation Network (SPN) structure. The cipher operates on a 64-bit data block, which is processed through multiple rounds of substitution, permutation, and key mixing. The master key is stored in a key register, from which round subkeys

are derived and applied at each round through an XOR operation with the state.

Formally, the encryption can be described as follows:

- The plaintext is first combined with the initial subkey through a bit-wise XOR (key mixing).
- Each round consists of the following transformations:
 - 1) Substitution layer: nonlinear transformation using 4×4 S-boxes.
 - 2) Permutation layer: diffusion of bits across positions to ensure interdependence.
 - 3) Key mixing: XOR of the state with the round subkey.
- After the final round, an additional subkey is applied to ensure that the last substitution layer cannot be trivially inverted by an adversary.

The generation of subkeys is handled by the key register, which updates after every round to provide fresh round keys. In our simplified model, the subkeys are assumed to be independent, but in practical ciphers they are derived systematically from the master key by a key scheduling algorithm.

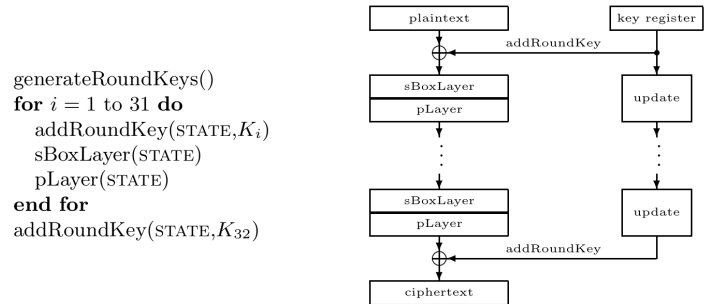


Fig. 4. Overall structure of the SPN cipher, showing the sequence of key mixing, S-box layer, and permutation, along with the key register updates.

III. ATTACK TYPES

Before introducing differential cryptanalysis, it is important to understand why simpler attack strategies are insufficient for SP-network ciphers. Brute-force search, although guaranteed to succeed, quickly becomes infeasible as key sizes grow, since the required computations increase exponentially. Likewise, linear cryptanalysis, which attempts to approximate nonlinear components with linear re-

lations, is largely ineffective when strong S-boxes and permutation layers are used.

These limitations motivate the use of differential cryptanalysis. By studying how input differences propagate through the substitution and permutation layers, this method reveals statistical patterns that can be exploited to recover key information. Thus, differential cryptanalysis provides a more practical and powerful approach where brute-force and linear methods fail.

A. Limitations of Brute-Force Attacks

Before introducing differential cryptanalysis, it is important to highlight why brute-force attacks are not a practical method for breaking modern block ciphers. A brute-force attack consists of exhaustively searching the entire key space by testing every possible key until the correct one is found. While this approach is theoretically guaranteed to succeed, its feasibility is determined entirely by the size of the key space.

For a key of length n bits, the adversary must attempt up to 2^n possible keys. As illustrated in Table 5, even moderate increases in key size cause the computational effort to grow exponentially. For example, a 32-bit key requires $2^{32} \approx 4.29 \times 10^9$ trials, which might be completed in a matter of minutes on modern hardware. However, a 64-bit key requires approximately $2^{64} \approx 1.84 \times 10^{19}$ trials, corresponding to billions of years of computation time, making it entirely impractical. At the extreme, a 1024-bit key would require on the order of $2^{1024} \approx 1.8 \times 10^{308}$ operations, which is many orders of magnitude longer than the age of the universe.

These estimates clearly demonstrate that brute-force attacks cannot be considered a realistic threat against well-designed ciphers with sufficiently large key sizes. As a result, modern cryptanalysis focuses on exploiting structural weaknesses in the cipher design, such as those revealed through *differential cryptanalysis*, rather than relying on exhaustive key search.

Bit length	Time Complexity	($2^n/2^{21}$)
2	2^2	1.9×10^{-6}
4	2^4	7.6×10^{-6}
8	2^8	1.2×10^{-4}
16	2^{16}	3.1×10^{-2}
32	2^{32}	2.0×10^3
64	2^{64}	8.8×10^7
128	2^{128}	1.7×10^{17}
256	2^{256}	1.2×10^{55}
512	2^{512}	1.3×10^{133}
1024	2^{1024}	1.8×10^{287}

Fig. 5. Brute-force time estimates for different key lengths, assuming a processing speed of 2^{21} operations per second.

IV. DIFFERENTIAL CRYPTANALYSIS

A. What is Differential Cryptanalysis?

Differential cryptanalysis is a form of *chosen-plaintext attack* that studies how differences in plaintext inputs affect the corresponding differences in ciphertext outputs. The central idea is to analyze the propagation of input differences through the rounds of a cipher and to exploit any non-random patterns or statistical biases that may arise. By observing these patterns, an adversary can gain information about the secret key.

B. Advantages of Differential Cryptanalysis

The power of differential cryptanalysis lies in two key aspects. First, the attacker attempts to identify **high-probability differential characteristics**, where certain input differences consistently lead to predictable output differences with greater frequency than chance. Second, these differential characteristics can be exploited to recover information about the subkeys, significantly reducing the complexity of key search compared to brute-force methods.

C. Basic Mechanism

The attack begins with the careful selection of plaintext pairs that differ by a specific value, denoted as

$$\Delta P = P_1 \oplus P_2.$$

Both plaintexts are encrypted under the same secret key, producing ciphertexts C_1 and C_2 . The difference in the outputs is then computed as

$$\Delta C = C_1 \oplus C_2,$$

where

$$C_1 = E(P_1, K), \quad C_2 = E(P_2, K).$$

By analyzing the relation between ΔP and ΔC across multiple pairs, the attacker can identify statistical tendencies that reveal information about the internal structure of the cipher and its key material.

D. Illustrative Dummy Example

Consider the following dummy example with an 8-bit key and two chosen plaintexts:

$$K = 01010010, \quad P_1 = 11010001, \quad P_2 = 10011101.$$

By encrypting these plaintexts and analyzing the differences in their ciphertexts, the attacker observes that the XOR difference cancels out part of the effect of the key mixing. This principle illustrates why differential analysis can reveal exploitable information about the cipher's design.

E. Step 1: S-box Difference Analysis

The first step in differential cryptanalysis involves analyzing the behavior of the cipher's S-boxes. Consider a 4×4 S-box with input $X = [X_1 X_2 X_3 X_4]$ and output $Y = [Y_1 Y_2 Y_3 Y_4]$. The goal is to determine how specific input differences ΔX propagate to output differences ΔY , and to measure the probability of each such transition.

F. Step 2: Difference Pairs

To perform this analysis, all possible input difference pairs ΔX are examined along with their resulting ΔY . The methodology is as follows:

- Choose an input value X' (out of 16 possible 4-bit values).
- Compute a paired input $X'' = X' \oplus \Delta X$.
- Determine the outputs $Y' = S(X')$ and $Y'' = S(X'')$.
- Compute the output difference $\Delta Y = Y' \oplus Y''$.

Since XOR differences are symmetric, each possible difference occurs multiple times, allowing probabilities to be derived from frequency counts. These differential distribution tables form the foundation of the attack, enabling the attacker to predict output differences with significant probability.

X	Y	ΔY		
		$\Delta X = 1011$	$\Delta X = 1000$	$\Delta X = 0100$
0000	1110	0010	1101	1100
0001	0100	0010	1110	1011
0010	1101	0111	0101	0110
0011	0001	0010	1011	1001
0100	0010	0101	0111	1100
0101	1111	1111	0110	1011
0110	1011	0010	1011	0110
0111	1000	1101	1111	1001
1000	0011	0010	1101	0110
1001	1010	0111	1110	0011
1010	0110	0010	0101	0110
1011	1100	0010	1011	1011
1100	0101	1101	0111	0110
1101	1001	0010	0110	0011
1110	0000	1111	1011	0110
1111	0111	0101	1111	1011

Fig. 6. Sample Difference Pairs of the S-box

		Output Difference															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
I	0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	2	0	0	0	2	0	2	4	0	4	2	0	0
	2	0	0	0	2	0	6	2	2	0	2	0	0	0	0	2	0
	3	0	0	2	0	2	0	0	0	0	4	2	0	2	0	0	4
P	4	0	0	0	2	0	0	6	0	0	2	0	4	2	0	0	0
	5	0	4	0	0	0	2	2	0	0	0	4	0	2	0	0	2
	6	0	0	0	4	0	4	0	0	0	0	0	0	2	2	2	2
	7	0	0	2	2	2	0	2	0	0	2	2	0	0	0	0	4
U	8	0	0	0	0	0	0	2	2	0	0	0	4	0	4	2	2
	9	0	2	0	0	2	0	0	4	2	0	2	2	2	0	0	0
	A	0	2	2	0	0	0	0	0	6	0	0	2	0	0	4	0
	B	0	0	0	0	0	2	0	2	0	0	0	0	0	2	0	2
D	C	0	2	0	0	2	2	2	0	0	0	0	2	0	6	0	0
	D	0	4	0	0	0	0	0	4	2	0	2	0	2	0	2	0
	E	0	0	2	4	2	0	0	0	6	0	0	0	0	0	2	0
	F	0	2	0	0	6	0	0	0	0	4	0	2	0	0	2	0

Fig. 7. Difference Distribution Table

G. Step 3: Probability Calculation

Once the input–output difference pairs have been determined for the S-box, the next step is to compute the probability of each differential transition. For a given input difference ΔX , all 16 possible inputs are considered, and the corresponding output differences ΔY are recorded.

For example:

- $\Delta X = 1011$ (hex B) $\Rightarrow \Delta Y = 0010$ occurs 8 times out of 16, giving a probability of $\frac{8}{16}$.
- $\Delta X = 1000$ (hex 8) $\Rightarrow \Delta Y = 1011$ occurs 4 times out of 16, giving a probability of $\frac{4}{16}$.
- $\Delta X = 0100$ (hex 4) $\Rightarrow \Delta Y = 1010$ never occurs, hence the probability is 0. In such cases, the pair is referred to as an **impossible differential**.

Ideally, a well-designed S-box would yield equal probabilities (approximately $2/16$ for a 4×4 S-box). Deviations from this uniform distribution create exploitable statistical biases that form the basis of differential cryptanalysis.

H. Step 4: Difference Distribution Table

To summarize these probabilities, a **Difference Distribution Table (DDT)** is constructed:

- The rows correspond to input differences ΔX .
- The columns correspond to output differences ΔY .
- Each entry represents the number of times a given ΔY occurs for the corresponding ΔX .

This tabular representation provides an efficient way to identify high-probability differential characteristics. For example, the pair $(\Delta X = B, \Delta Y = 2)$ may occur 8 out of 16 times, giving a noticeably higher probability than expected by chance. Conversely, many pairs have zero occurrences, which can be leveraged in an **impossible differential attack**, where keys inconsistent with zero-probability characteristics can be discarded immediately.

I. Step 5: Interpretation of the Tables

The Difference Distribution Table serves two main purposes:

- It provides a detailed mapping between input and output difference pairs of the S-box. This allows cryptanalysts to observe the statistical structure underlying the nonlinear transformation.
- It summarizes the occurrence counts of each differential transition, making it easier to identify high-probability trails. These trails guide the construction of differential characteristics across multiple rounds of the cipher.

Thus, the DDT is a powerful tool for analyzing the strength of an S-box and for mounting practical differential attacks.

J. Key Recovery Using Differential Characteristics

The ultimate goal of differential cryptanalysis is to recover the secret key. Once high-probability characteristics have been identified, the attack proceeds as follows:

- Candidate keys are tested against the observed differential characteristics.
- For each tested key, a counter is incremented whenever the characteristic holds for a chosen plaintext pair.
- After processing a sufficient number of plaintext-ciphertext pairs, the correct key will have

the highest counter value, as it best matches the differential pattern.

In this way, differential cryptanalysis reduces the complexity of key recovery far below brute-force levels by exploiting non-uniform differential properties of the cipher's components.

K. Differential Distribution Tables (DDTs) for Multiple S-Boxes

To quantify how input differences propagate through the substitution layer, we compute the *Difference Distribution Table* (DDT) for each S-box. For a 4×4 S-box $S : \{0, \dots, 15\} \rightarrow \{0, \dots, 15\}$ and an input difference $\Delta X \in \{0, \dots, 15\}$, the DDT entry at $(\Delta X, \Delta Y)$ counts

$$\#\{x \in \{0, \dots, 15\} \mid S(x) \oplus S(x \oplus \Delta X) = \Delta Y\}.$$

Thus, each row (fixed ΔX) sums to 16. Large counts indicate *high-probability* transitions; zero entries indicate *impossible differentials*. These statistics are the basis for constructing round characteristics and for pruning keys inconsistent with zero-probability trails.

a) Algorithm and Complexity.: The DDT can be computed by enumerating all ordered input pairs (i, j) , accumulating counts for $(\Delta X = i \oplus j, \Delta Y = S[i] \oplus S[j])$. For 4-bit S-boxes this is $16^2 = 256$ operations per S-box (i.e., negligible).

b) Reference Implementation (C++).: The listing below computes the Difference Distribution Table (DDT) for three 4-bit S-boxes (S_0, S_1, S_2) by enumerating all ordered input pairs (i, j) and accumulating counts for $dX = i \oplus j$ and $dY = S[i] \oplus S[j]$. Each row (fixed dX) sums to 16.

```
#include <iostream>
using std::cout; using std::endl;

void Count_Diff(const int S[16], int C[16][16]) {
    for (int i = 0; i < 16; ++i)
        for (int j = 0; j < 16; ++j) {
            const int dX = i ^ j;
            const int dY = S[i] ^ S[j];
            ++C[dX][dY];
        }
}

void Show_Counts(const char* name, int C[16][16]) {
    const char H[] = "0123456789ABCDEF";
    cout << "Input Difference | Output Difference\n";
    for (int x = 0; x < 16; ++x) {
        cout << name << " | ";
        for (int y = 0; y < 16; ++y) cout << H[y] << " | ";
        cout << "\n";
    }
}
```



```

cout << "\n";
for (int dx = 0; dx < 16; ++dx) {
    cout << ' ' << H[dx] << '|';
    int rowsum = 0;
    for (int dy = 0; dy < 16; ++dy) {
        cout << ' ' << C[dx][dy] << '|';
        rowsum += C[dx][dy];
    }
    // sanity check: each row should sum to 16
    // cout << " (sum=" << rowsum << ')';
    cout << "\n";
}

int main() {
    const int S_0[16] = {
        0xC, 0x5, 0x6, 0xB, 0x9, 0x0, 0xA, 0xD,
        0x3, 0xE, 0xF, 0x8, 0x4, 0x7, 0x1, 0x2
    };
    const int S_1[16] = {
        0xE, 0x4, 0xD, 0x1, 0x2, 0xF, 0x2, 0xB,
        0x8, 0x3, 0xA, 0x6, 0xC, 0x5, 0x9, 0x0
    };
    const int S_2[16] = {
        0x0, 0xB, 0xC, 0x5, 0x6, 0x1, 0x9, 0xA,
        0x3, 0xF, 0xE, 0x8, 0xD, 0x4, 0x2, 0x7
    };

    int C0[16][16] = {0}, C1[16][16] = {0},
        C2[16][16] = {0};

    Count_Diff(S_0, C0);
    Count_Diff(S_1, C1);
    Count_Diff(S_2, C2);

    Show_Counts("S_0", C0); cout << "\n";
    Show_Counts("S_1", C1); cout << "\n";
    Show_Counts("S_2", C2);
    return 0;
}

```

Listing 1. DDT computation for 4-bit S-boxes

Including the Tables. The computed Difference Distribution Tables (DDTs) obtained from the reference implementation can be included in the report as figures. Each table summarizes the distribution of output differences for a fixed input difference, and thus highlights high-probability transitions (large counts) as well as impossible differentials (zero entries). This allows the results for different S-boxes (S_0, S_1, S_2) to be clearly compared and analyzed within the context of differential cryptanalysis.

Input Difference	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	4	0	0	0	4	0	4	0	0	0	4	0	0
2	0	0	0	2	0	4	2	0	0	0	2	0	2	2	2	0
3	0	2	0	2	2	0	4	2	0	0	2	2	0	0	0	0
4	0	0	0	0	0	4	2	2	0	2	2	0	2	0	2	0
5	0	2	0	0	2	0	0	0	0	2	2	2	4	2	0	0
6	0	0	2	0	0	0	2	0	2	0	0	4	2	0	0	4
7	0	4	2	0	0	0	2	0	2	0	0	0	2	0	0	4
8	0	0	0	2	0	0	0	2	0	2	0	4	0	2	0	4
9	0	0	2	0	4	0	2	0	2	0	0	0	2	0	4	0
A	0	0	2	2	0	4	0	0	2	0	2	0	0	2	2	0
B	0	2	0	0	2	0	0	0	4	2	2	2	0	2	0	0
C	0	0	2	0	0	4	0	2	2	2	2	0	0	0	2	0
D	0	2	4	2	2	0	0	2	0	0	2	2	0	0	0	0
E	0	0	2	2	0	0	2	2	2	2	0	0	2	2	0	0
F	0	4	0	0	4	0	0	0	0	0	0	0	0	0	4	4

Fig. 8. DDT for S_0 (Input Difference vs. Output Difference counts).

Input Difference	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	6	2	2	4	2	0	0
2	2	0	2	2	2	8	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	6	0	0	4	2	2	2
4	0	0	0	2	2	0	4	0	0	0	2	2	2	0	0	2
5	0	2	0	2	0	0	4	0	0	0	2	0	0	2	0	4
6	0	2	0	4	0	0	2	0	0	0	0	0	2	0	2	4
7	0	0	2	2	0	2	2	0	2	0	4	0	0	0	0	2
8	0	0	0	0	0	0	2	6	0	0	2	4	0	0	2	0
9	0	0	4	2	0	0	0	2	0	0	0	4	2	2	0	0
A	0	0	4	0	2	2	0	0	0	0	0	2	0	0	4	2
B	0	0	2	0	0	0	2	4	2	2	0	0	0	0	4	0
C	0	4	2	0	2	0	0	2	0	2	0	2	2	2	0	0
D	0	4	0	0	2	0	0	2	4	0	0	2	0	2	0	0
E	0	2	0	0	4	0	0	2	4	2	2	0	0	0	0	0
F	0	2	0	2	2	2	0	0	2	0	0	0	0	4	2	0

Fig. 9. DDT for S_1 . High-count cells indicate high-probability transitions; zeros are impossible differentials.

Input Difference	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	0	2	2	2	0	4	0	2	2	0	0	0
2	0	0	0	2	0	0	0	2	0	0	0	2	2	2	2	4
3	0	2	0	0	0	2	2	2	2	0	2	2	2	0	0	0
4	0	0	0	0	0	2	2	0	0	0	2	2	2	0	2	4
5	0	2	2	0	0	0	2	2	0	2	2	0	2	2	0	0
6	0	4	0	2	2	0	0	0	2	2	2	0	2	0	0	0
7	0	0	2	2	2	2	0	0	0	0	4	0	0	4	0	0
8	0	0	2	2	2	2	0	0	0	0	4	0	4	0	0	0
9	0	0	2	0	2	0	0	0	4	0	0	2	2	0	2	2
A	0	0	0	2	4	0	2	0	0	0	2	0	0	0	4	2
B	0	2	0	4	0	2	2	2	2	0	0	0	0	2	0	0
C	0	0	4	0	0	2	0	2	0	0	0	0	2	4	2	2
D	0	2	2	0	4	0	2	2	0	2	0	2	0	0	0	0
E	0	4	2	0	0	2	0	0	2	2	2	0	2	0	0	0
F	0	0	0	0	0	0	2	2	4	4	0	0	0	0	2	2

Fig. 10. DDT for S_2 .

		Output Difference															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Input Difference	0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	2	0	0	0	2	0	2	4	0	4	2	0	0
	2	0	0	0	2	0	6	2	2	0	2	0	0	0	0	2	0
	3	0	0	2	0	2	0	0	0	0	4	2	0	2	0	0	4
	4	0	0	0	2	0	0	6	0	0	2	0	4	2	0	0	0
	5	0	4	0	0	0	2	2	0	0	0	4	0	2	0	0	2
	6	0	0	0	4	0	4	0	0	0	0	0	2	2	2	2	2
	7	0	0	2	2	2	0	2	0	0	2	2	0	0	0	0	4
	8	0	0	0	0	0	0	2	2	0	0	0	4	0	4	2	2
	9	0	2	0	0	2	0	0	4	2	0	2	2	2	0	0	0
	A	0	2	2	0	0	0	0	0	6	0	0	2	0	0	4	0
	B	0	0	8	0	0	2	0	2	0	0	0	0	0	2	0	2
	C	0	2	0	0	2	2	2	0	0	0	2	0	6	0	0	0
	D	0	4	0	0	0	0	0	4	2	0	2	0	2	0	2	0
	E	0	0	2	4	2	0	0	0	6	0	0	0	0	0	2	0
	F	0	2	0	0	6	0	0	0	0	4	0	2	0	0	2	0

Fig. 11. Illustrative DDT view highlighting a high-count entry (e.g., $\Delta X = B$, $\Delta Y = 2$ with count 8).

c) Observations.:

- **Row sums.** For every fixed ΔX , the counts over all ΔY sum to 16. The first row ($\Delta X = 0$) concentrates entirely at $\Delta Y = 0$ (value 16).
- **High-probability differentials.** Entries with counts $\gg 2$ (for a 4-bit S-box) indicate biases exploitable for differential trails across rounds.
- **Impossible differentials.** Zero entries can be used to eliminate key candidates early (impossible differential attacks).
- **Bijectivity note.** DDT computation does not require S to be bijective; however, decryption in an SPN does. If an S-box is non-bijective, it weakens invertibility and may harm security even if the DDT looks balanced.

REFERENCES

- [1] H. M. Heys, “A tutorial on linear and differential cryptanalysis,” *Cryptologia*, vol. 26, no. 3, pp. 189–221, 2002. doi: <https://doi.org/10.1080/01611190208984253>
- [2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010.
- [3] E. Biham and A. Shamir, “Differential cryptanalysis of DES-like cryptosystems,” *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1990. doi: <https://doi.org/10.1007/BF00196718>
- [4] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Viskellsoe, “PRESENT: An ultra-lightweight block cipher,” in *Cryptographic Hardware and Embedded Systems – CHES 2007*, P. Paillier and I. Verbauwhede, Eds. Berlin, Heidelberg: Springer, 2007, pp. 450–466. doi: https://doi.org/10.1007/978-3-540-74735-2_31
- [5] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996.
- [6] D. R. Stinson and M. B. Paterson, *Cryptography: Theory and Practice*, 4th ed. CRC Press, 2018.

- [7] E. Biham and A. Shamir, “Differential cryptanalysis of Feistel ciphers,” in *Advances in Cryptology – CRYPTO ’91*, LNCS 576, J. Feigenbaum, Ed. Springer, 1992, pp. 2–21.
- [8] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*. Springer, 2002.
- [9] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, “The SIMON and SPECK lightweight block ciphers,” in *Proceedings of the 52nd Annual Design Automation Conference (DAC)*, ACM, 2015.